

Análise de Sorts

Enrico Panico Laguna

Felipe do Amparo Lauria

Instituto de Biociências, Letras e Ciências Exatas, Unesp -
Universidade Estadual Paulista, 15054-000, São José do Rio Preto - SP,
Brasil.

Outubro 2024

1 Introdução

Este relatório apresenta uma análise comparativa de desempenho entre diferentes algoritmos de ordenação aplicados a vetores de diversos tamanhos e configurações de ordem (aleatórios, ordenados e inversamente ordenados). Foram implementados e testados os algoritmos Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort, Heap Sort, Radix Sort, Counting Sort, Bucket Sort e Shell Sort, utilizando as linguagens C. A partir dos resultados de tempo de execução em cada cenário, buscou-se entender a eficiência prática dos algoritmos em relação à sua complexidade teórica, destacando a adequação de cada método a diferentes tipos de dados e tamanhos de vetores.

2 Análise

No cenário de vetores já ordenados, o Quick Sort apresenta um desempenho notável, beneficiando-se do particionamento otimizado, com complexidade média de $O(n \log n)$. O Insertion Sort também mostra significativa melhora nesse contexto, dado que é eficiente em dados quase ordenados, o que reduz seu tempo de execução, enquanto o Selection Sort continua ineficiente, uma vez que seu processo de seleção mínima não aproveita a ordenação existente, mantendo complexidade $O(n^2)$ e a mesma ineficiência é notada no Bubble Sort. O Merge Sort e o Heap Sort permanecem estáveis, com desempenho sólido e previsível em $O(n \log n)$. O Bucket Sort, por sua vez, é especialmente eficiente em vetores ordenados, pois a baixa redistribuição entre os buckets reduz o tempo de execução. Counting Sort e Radix Sort mantêm sua robustez em dados ordenados, executando com complexidade linear $O(n)$ e garantindo eficiência mesmo em grandes volumes de dados inteiros.

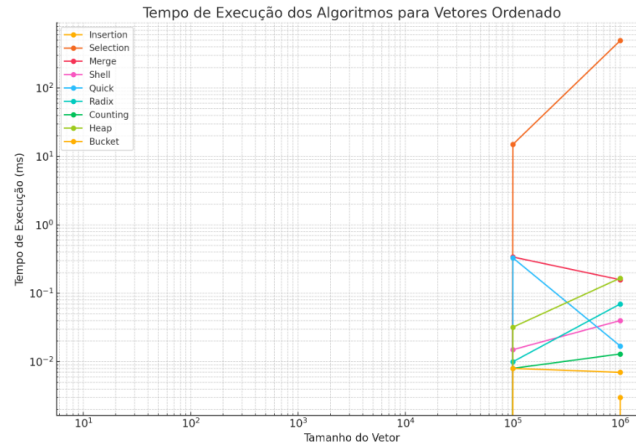


Figura 1: Desempenho em vetores Ordenados

Para vetores inversamente ordenados, o Bubble Sort o Insertion Sort e o Selection Sort mostram-se novamente ineficazes, com tempos de execução altos devido à complexidade $O(n^2)$, sendo fortemente impactados pela ordem inversa dos dados. O Quick Sort, embora apresente uma degradação leve, permanece razoavelmente eficiente na maioria dos casos, mas seu pior caso, $O(n^2)$, é atingido com maior probabilidade em vetores ordenados de forma inversa. Merge Sort e Heap Sort mantêm desempenho estável e eficaz, com complexidade $O(n \log n)$, mesmo quando os dados estão em ordem inversa. O Bucket Sort também se mantém competitivo, dependendo da redistribuição eficaz dos valores nos buckets. Tanto o Counting Sort quanto o Radix Sort continuam a mostrar boa performance, apesar de que, no caso de Counting Sort, a dispersão excessiva dos valores pode exigir mais espaço, mas ainda mantendo o tempo linear $O(n)$ nos casos ideais.

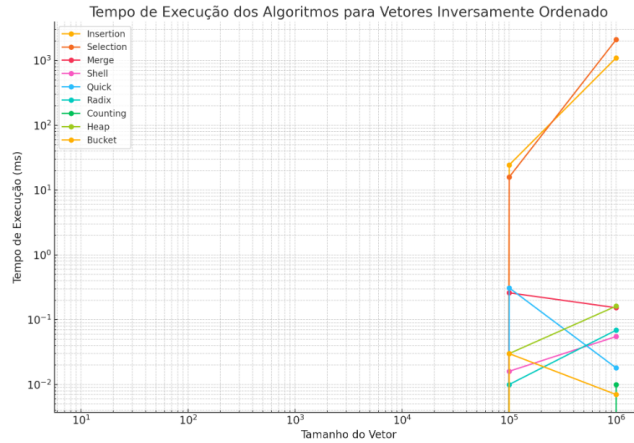


Figura 2: Desempenho em vetores Inversamente Ordenados

Quando aplicados a vetores desordenados, Insertion Sort e Selection Sort mostram crescimento expressivo no tempo de execução, especialmente em vetores com 10^5 e 10^6 elementos, devido à sua complexidade $O(n^2)$. Em contrapartida, algoritmos com complexidade $O(n \log n)$, como Merge Sort, Quick Sort e Heap Sort, demonstram desempenho mais consistente, sendo mais adequados para grandes vetores sem ordem específica. O Bucket Sort se destaca, executando com tempos relativamente baixos para grandes vetores desordenados, pois sua eficiência aumenta quando os dados se distribuem bem entre os buckets. Counting Sort e Radix Sort são os mais eficientes nesse cenário, com complexidade $O(n)$, mostrando-se ideais para grandes conjuntos de dados com valores inteiros limitados, especialmente quando os valores não apresentam alta variabilidade.

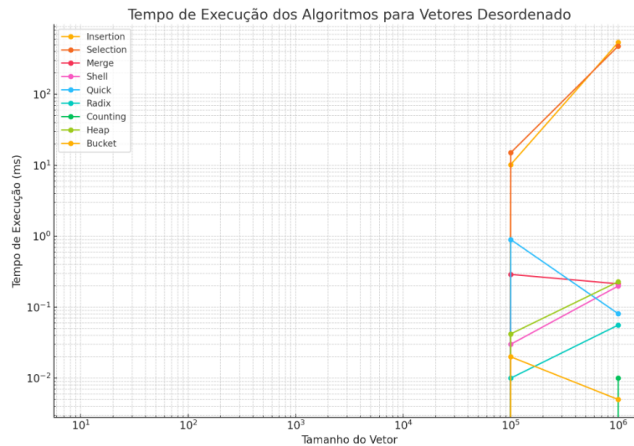


Figura 3: Desempenho em vetores Desordenados

3 Conclusão

A análise comparativa de desempenho dos diferentes algoritmos de ordenação revelou resultados significativos que refletem as complexidades teóricas e as eficiências práticas de cada método em diversas configurações de dados. Os algoritmos de ordenação como Quick Sort e Merge Sort mostraram-se consistentes e eficazes, especialmente em vetores de tamanhos grandes, enquanto métodos simples como Bubble Sort, Insertion Sort e Selection Sort apresentaram desempenho insatisfatório em cenários de vetores ordenados e inversamente ordenados, sendo mais adequados apenas para conjuntos de dados pequenos ou quase ordenados. O Bucket Sort destacou-se em particular em situações em que os dados apresentavam uma distribuição uniforme, mostrando que sua eficácia pode superar a de algoritmos mais complexos quando aplicados corretamente. Por outro lado, Counting Sort e Radix Sort mostraram-se os mais eficientes em cenários de dados inteiros limitados, destacando sua robustez em grandes volumes de dados.